

\$ Computação de Alto Desempenho - Professora Lícia Sales

Otimização de um pipeline de pré-processamento de imagens

CIFAR-10, escala de cinza e detecção de bordas com Sobel: cinco implementações medidas lado a lado.

Giulia Roggero

Maria Luiza Sevilha Seraphico

Thais Sabaini

\$ 0 problema

Pré-processar um dataset inteiro exige otimizações de alto desempenho

A tarefa é simples de escrever, cara de executar e fácil de validar, o que permite comparar implementações sem ambiguidade sobre a corretude.

01

Cada imagem é **independente** das demais: o problema é paralelo por natureza.

02

A baseline é trivial de escrever em Python e péssima de performance, o que dá uma referência interessante.

03

A corretude é verificável pixel a pixel: toda versão precisa devolver exatamente a mesma saída.

numba

O pipeline tem quatro etapas e a última domina o custo

Toda implementação comparada neste trabalho executa exatamente estas quatro etapas, na mesma ordem e com a mesma aritmética.

`load_dataset()`

Ler os arquivos
`data_batch` em pickle.

→

`reconstruct_images()`

Remontar cada linha de
3.072 bytes em $32 \times 32 \times 3$.

→

`grayscale()`

Combinação ponderada
dos canais R, G e B.

→

`sobel()`

Dois kernels 3×3 ,
magnitude do gradiente e
normalização.

A saída é um par de arrays por lote: as imagens em escala de cinza e o mapa de bordas, ambos em `float32` de 32×32 .

\$ Base de dados

A CIFAR-10 chega como 3.072 bytes por linha, não como imagem

Cada batch é um pickle com uma matriz 10.000×3.072 em `uint8`, o que obriga uma reconstrução antes de qualquer processamento.

60.000 imagens de $32 \times 32 \times 3$ em 10 classes, sendo 50.000 de treino em cinco arquivos `data_batch` e 10.000 de teste.

O split de treino define o teto dos benchmarks: todas as curvas deste deck vão de 100 a 50.000 imagens.

Cada linha traz 1.024 valores de R, depois 1.024 de G, depois 1.024 de B, e não os três canais entrelaçados por pixel.

layout de uma linha

R · 1024

G · 1024

B · 1024

`reshape(3, 32, 32) → transpose(1, 2, 0)`

A transposição devolve uma *view*, não uma cópia.

\$ Baseline

A baseline percorre pixel a pixel e vizinho a vizinho em Python puro

É a tradução literal da definição do operador Sobel, sem nenhuma biblioteca numérica, e serve como referência de tempo e de corretude.

baseline.py · sobel_single()

```
for y in range(32):
    for x in range(32):
        gx = gy = 0.0
        for j in range(-1, 2):
            for i in range(-1, 2):
                v = img[_clamp(y+j)][_clamp(x+i)]
                gx += v * KX[j+1][i+1]
                gy += v * KY[j+1][i+1]
        out[y][x] = (gx*gx + gy*gy) ** 0.5
```

A linha destacada é o ponto do slide: `_clamp()` é uma chamada de função Python executada duas vezes para cada um dos nove vizinhos.

Nada aqui está errado do ponto de vista algorítmico: a complexidade é a mesma das versões rápidas.

Com 50.000 imagens, esta versão leva **228,22s**.

\$ Profiling

92,6% do tempo da baseline está dentro de `sobel_single()`

Perfil de 5.000 imagens com cProfile: o Sobel domina, o grayscale é um detalhe e a leitura dos dados é irrelevante.

```
load_dataset() · 0,15% · 0,056 s  
rgb_to_grayscale_single() · 7,3% · 2,65 s
```

92,6%

`sobel_single()`

33,72 s

FUNÇÃO	TEMPO	%
<code>sobel_single()</code>	33,72 s	92,6%
<code>rgb_to_grayscale_single()</code>	2,65 s	7,3%
<code>load_dataset() + pickle.load</code>	0,056 s	0,15%

O total sob o profiler é de 36,4s contra 24s a 28s sem instrumentação: o cProfile cobra caro justamente por medir chamadas de função, e é disso que este programa é feito. A leitura do gráfico é direta: otimizar qualquer coisa que não seja o Sobel não muda o resultado.

\$ Diagnóstico

0 gargalo são 92 milhões de chamadas de função

Só `_clamp()` é chamada 92.160.000 vezes em 5.000 imagens e consome 7,74 s de tempo próprio.

$$\begin{array}{c} 2 \\ \text{clamps} \end{array} \times \begin{array}{c} 9 \\ \text{vizinhos} \end{array} \times \begin{array}{c} 1.024 \\ \text{pixels} \end{array} \times \begin{array}{c} 5.000 \\ \text{imagens} \end{array} = \begin{array}{c} 92.160.000 \\ \text{chamadas} \end{array}$$

Cada chamada custa criação de frame, resolução de nomes e boxing de floats. A aritmética em si (duas multiplicações e uma soma) é a menor parte do trabalho.

O alvo, portanto, não é fazer a conta mais rápido, e sim tirar o interpretador do caminho: eliminar o laço, compilá-lo, ou executá-lo em muitas unidades ao mesmo tempo.

Duas otimizações reduzem o custo por operação e duas somam unidades

As quatro atacam o mesmo gargalo por caminhos diferentes, ordenadas aqui por complexidade de implementação.

OTIMIZAÇÃO	COMPLEXIDADE	IDEIA	ATACA O GARGALO POR
1 · <code>vectorized</code> (NumPy)	baixa	reescreve o pipeline em operações de array	remover o laço do Python
2 · <code>numba</code>	média	@ <code>njit</code> compila o mesmo código para nativo	compilar o laço
3 · <code>multiprocessing</code>	média-alta	distribui imagens entre processos	somar núcleos de CPU
4 · <code>gpu</code> (PyTorch/CUDA)	alta	reescreve como operações de tensor	paralelismo massivo

Leitura da tabela: as duas primeiras tornam cada operação mais barata; as duas últimas mantêm o custo por operação e aumentam quantas acontecem ao mesmo tempo. A diferença entre esses dois eixos organiza todo o resto do trabalho.

Vetorização

O laço vira uma soma de doze fatias deslocadas do array inteiro, executadas em C dentro do NumPy: ganho de **118x** sobre a baseline.

vectorized.py

```
p = np.pad(gray, ((0,0),(1,1),(1,1)), 'edge')
gx = np.zeros_like(gray)
for j in range(3):
    for i in range(3):
        if KX[j][i]:
            gx += KX[j][i] * p[:, j:j+32, i:i+32]
edges = np.sqrt(gx*gx + gy*gy)
```

A linha destacada é o ponto do slide: o `np.pad` resolve a borda de uma vez e permite tratar as 32×32 posições como fatias, sem condicional por pixel.

O custo escondido está no +=: cada um dos doze deslocamentos materializa um array temporário de N×32×32.

Em volume alto a operação passa a ser limitada por banda de memória, e o custo marginal por imagem cresce.

Numba

O código continua sendo laços explícitos, quase idêntico à baseline, mas compilado para nativo: ganho de **319x** sobre a baseline.

numba_version.py

```
@njit(cache=True)
def process_images_numba(images_rgb):
    for idx in range(images_rgb.shape[0]):
        _grayscale_into(images_rgb[idx], grays[idx])
        _sobel_into(grays[idx], edges[idx])
    return grays, edges
```

A linha destacada é o ponto do slide: um decorador transforma o mesmo laço em código de máquina, sem reescrever o algoritmo.

Fusão de laços: grayscale, gradiente, magnitude e normalização acontecem em uma passada por imagem, sem arrays temporários.

Localidade de cache: uma imagem de 32×32 cabe em L1/L2 e é processada inteira antes da próxima.

Em troca, há um **custo fixo de compilação**

Multiprocessing

Distribuir a baseline entre processos rende um ganho de apenas **2,2x**, o pior retorno das quatro estratégias.

```
multiprocessing_version.py
```

```
chunks = np.array_split(images, n_workers)
with Pool(n_workers) as pool:
    results = pool.map(_process_chunk, chunks)
grays = np.concatenate([r[0] for r in results])
```

A linha destacada é o ponto do slide: cada chunk atravessa a fronteira entre processos serializado em pickle, na ida e na volta.

O trabalho enviado a cada worker continua sendo o mesmo código interpretado da baseline: quatro processos executando código lento.

Diagnóstico: multiplicar unidades de execução sem reduzir o custo por operação tem retorno baixo, e a serialização come parte do que se ganha.

GPU

O pipeline vira uma sequência de operações de tensor executadas em milhares de threads: gera um ganho de **412x** sobre a baseline.

gpu_version.py

```
t = torch.from_numpy(images).to(device)
gray = (t * WEIGHTS).sum(dim=-1)
gx = F.conv2d(F.pad(gray, (1,1,1,1), 'replicate'), KX)
edges = torch.sqrt(gx*gx + gy*gy)
return edges.cpu().numpy()
```

A linha destacada é o ponto do slide: antes de qualquer cálculo, os dados precisam atravessar o barramento até a memória da GPU e voltar no fim.

Esse custo fixo é o motivo de a GPU ser a mais lenta das versões rápidas em volumes pequenos.

É também a **versão mais distante do código original**: quem for manter precisa entender tensores, convolução e dispositivos.

Compilação e execução são custos de naturezas diferentes

Medir as duas juntas como se fossem uma confunde a conclusão do trabalho

PARCELA	COMPORTAMENTO	ORDEM DE GRANDEZA	COMO É REPORTADA
compilação JIT	fixa, independente do volume	≈ 140 s, uma vez	separada, como custo de setup
execução nativa	linear no volume	14,3 μ s por imagem	resultado principal (regime quente)

DECISÃO ADOTADA

O **regime quente** é o resultado principal e a compilação é reportada à parte, do mesmo jeito que não se contabiliza o `import torch` nem a inicialização do contexto CUDA no custo por imagem da GPU. Reportar o JIT dentro do tempo de processamento e não reportar o setup da GPU seria comparar as duas versões com régua diferentes.

\$ Custo de setup

0 custo do JIT é fixo: cerca de 140 s, com 100 ou com 5.000 imagens

Recompilando a cada ponto, a curva não cresce com o volume: isso só faz sentido se quase todo o tempo for compilação.



Com 5.000 imagens, o Numba quente custa **0,078s** contra **140,02s** frio.

O perfil confirma a leitura da curva: dos 192,3s de uma execução a frio, 192,18s são compilação.



Lido corretamente, este é o argumento mais forte a favor do Numba: o que ele cobra é uma vez só, e `cache=True` amortiza até isso entre execuções.

\$ Medição

Com 5.000 imagens, o desvio padrão do Numba é maior que a sua média

Três repetições em um único ponto de volume não separam as duas versões rápidas e escondem o efeito da compilação dentro da média.

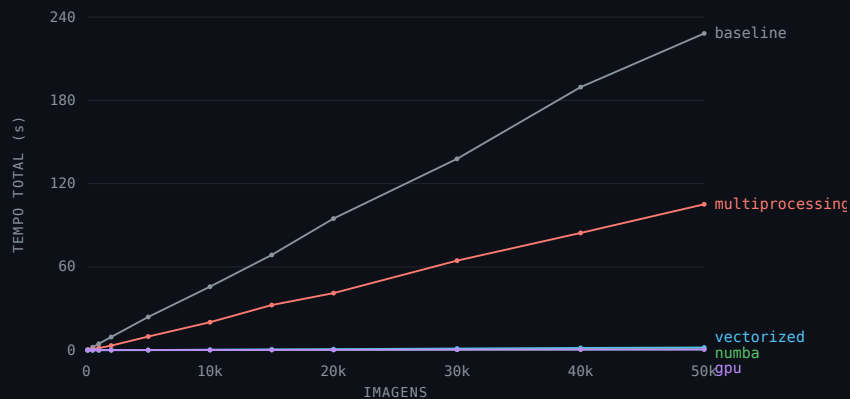
VERSÃO	MÉDIA DE 3 EXECUÇÕES
baseline	28,24s
multiprocessing	12,55s
vectorized	0,153s
numba	0,236s ± 0,249s
gpu	0,0938s

O QUE O DESVIO REVELA

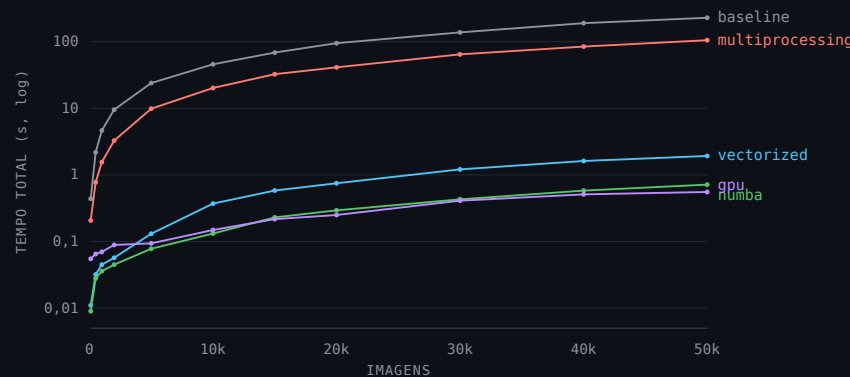
A primeira execução pagou compilação e as duas seguintes não. A média de três repetições nessas condições não descreve nenhuma das três execuções.

Nesse ponto o Numba parece *perder* para a vetorizada. É esse artefato de medição que motiva medir a escalabilidade em regime quente, ponto a ponto.

Em escala linear só a baseline aparece; em log as cinco versões se separam



Escala linear: a baseline domina o eixo e achata as outras quatro contra a base do gráfico.



Escala log: três ordens de grandeza separando baseline e GPU, e as três curvas rápidas finalmente distinguíveis.

118x vectorized **319x** numba **412x** gpu **2,2x** multiprocessing

speedup sobre a baseline com 50.000 imagens

\$ Custo marginal

0 custo por imagem da vetorizada cresce com o volume; o do Numba não

Dividindo o tempo total pelo número de imagens, as curvas mostram como cada versão se comporta ao crescer o lote.



A vetorizada materializa doze arrays temporários por lote; a partir de alguns milhares de imagens ela passa a ser limitada por banda de memória.

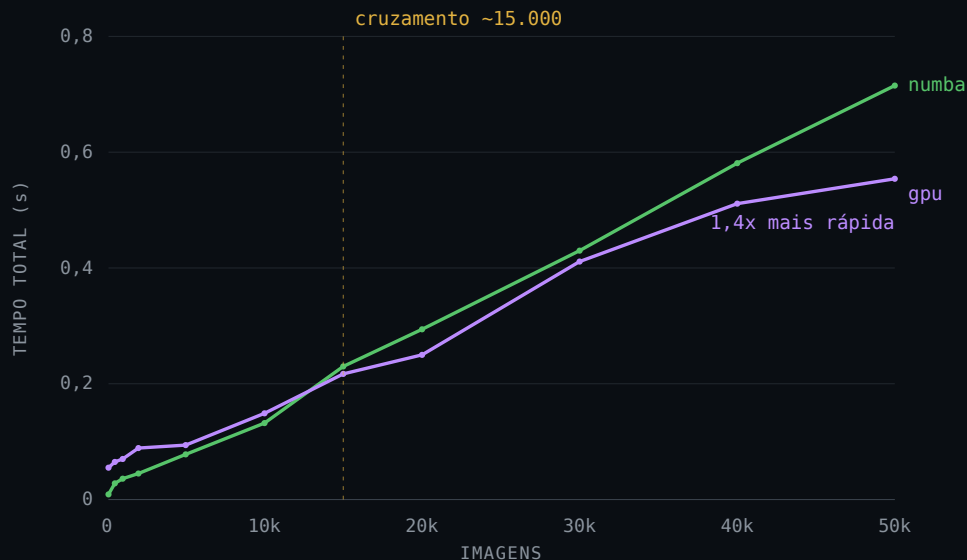
O Numba processa uma imagem por vez dentro do cache, então o custo por imagem se estabiliza.

CUSTO MARGINAL, AJUSTE LINEAR $n \geq 5.000$

vetorizada	49,38 μs
numba	20,55 μs
gpu	12,90 μs

\$ Ponto de virada

A GPU passa o Numba a partir de ~15.000 imagens e a distância cresce



O melhor resultado em CPU perde para a GPU exatamente na faixa em que o problema deixa de ser um exercício e passa a ser um lote de produção.

Aceitar isso como conclusão significa aceitar que processar imagens em escala exige hardware dedicado.

De onde vem a vantagem da GPU?

\$ Hipótese

A GPU não calcula melhor: ela calcula muitas imagens ao mesmo tempo

A aritmética é a mesma e as saídas são idênticas, então a diferença de tempo vem de quantas unidades trabalham em paralelo.

CONSTATAÇÃO

O Numba, por melhor que esteja compilado, percorre o laço em **uma única thread**, com os outros núcleos parados.

JÁ ESTABELECIDO

O problema é **paralelo por natureza**: cada imagem é independente das demais, sem dependência entre iterações.

OBJEÇÃO

O multiprocessing já tentou explorar isso e rendeu 2,2x, mas por serializar chunks entre processos e paralelizar código interpretado.

RESPOSTA

Nenhuma das duas objeções se aplica a threads sobre código já compilado, com memória compartilhada e sem pickle.

| E se o laço que já está compilado for **distribuído** entre os núcleos?

\$ Implementação

range vira prange: a mudança é de uma linha

O corpo do laço é o mesmo da versão serial; o que muda é quem executa cada iteração.

```
numba_parallel.py
```

```
from numba import njit, prange
```

```
@njit(parallel=True, cache=True)
```

```
def process_images_numba_parallel(images_rgb):
```

```
    n = images_rgb.shape[0]
```

```
    grays = np.empty((n, 32, 32), dtype=np.float32)
```

```
    edges = np.empty((n, 32, 32), dtype=np.float32)
```

```
    for idx in prange(n):
```

```
        _grayscale_into(images_rgb[idx], grays[idx])
```

```
        _sobel_into(grays[idx], edges[idx])
```

1. A aritmética por pixel é idêntica à da versão serial, de propósito: qualquer diferença de tempo é **atribuível só ao paralelismo**.

2. Cada iteração escreve apenas em `grays[idx]` e `edges[idx]`: sem escrita compartilhada e sem redução em ponto flutuante entre threads.

3. O número de threads é controlável por `numba.set_num_threads`, o que permite medir escalabilidade em vez de aceitar o default.

Custo: compilar com `parallel=True` é bem mais lento que o `njit` simples. Vale a mesma régua da metodologia: é setup, não processamento.

\$ Corretude

A saída é idêntica à da versão serial

Um ganho de tempo só conta como otimização se o resultado não mudar e em ponto flutuante isso não é automático.

Paralelizar sobre um índice cujas iterações escrevem em regiões disjuntas não altera a ordem de nenhuma operação aritmética.

Não há redução em ponto flutuante entre threads, que é o caso em que a ordem de soma mudaria o último bit do resultado.

A verificação é automatizada em `benchmark/verify_pipeline.py` e cobre todas as versões, não só a paralela.

CRITÉRIO DE ACEITAÇÃO

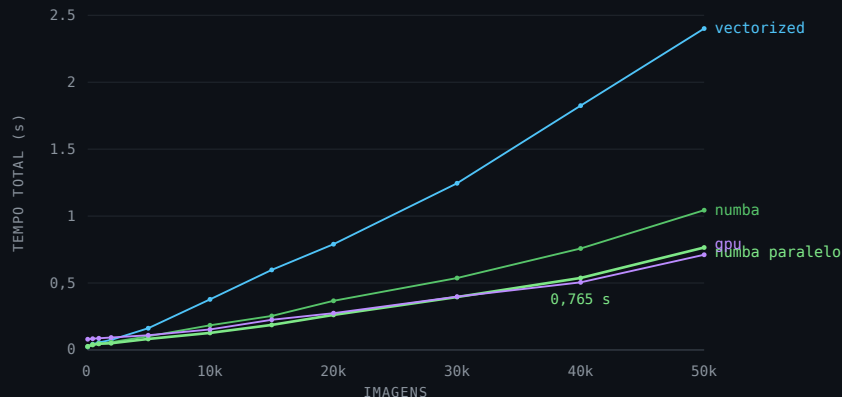
`np.array_equal(serial, paralelo)`

Igualdade exata, não `allclose`: nenhuma tolerância numérica foi usada para aceitar a versão paralela.

\$ Resultado

0 prange rende 1,37x sobre o Numba serial com 50.000 imagens

As quatro versões rápidas remedidas na mesma sessão, para que a comparação seja legítima: 1,044s caem para 0,765s.



Tempo total: o numba paralelo fica abaixo do serial em toda a faixa e acompanha a GPU de perto.

O ganho cresce com o volume: em lotes pequenos o overhead de threads domina, e a partir de ~5.000 imagens ele se paga.

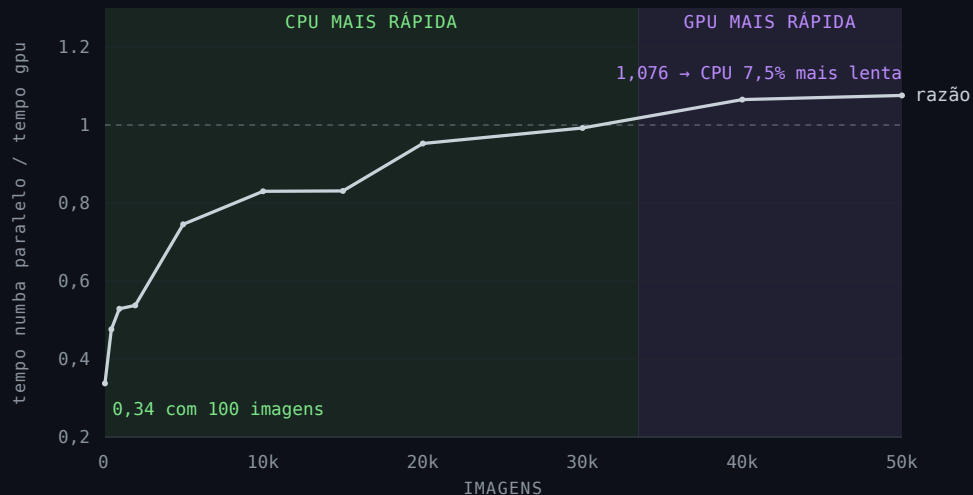
1,37x vs numba serial

3,14x vs vetorizada

0,765s com 50.000 imagens

\$ CPU contra GPU

A CPU lidera até 30.000 imagens e perde por 7,5% no maior volume



Abaixo da linha de 1,0 a CPU é mais rápida. O numba paralelo vence em todos os volumes até 30.000 imagens; a GPU só recupera a liderança em 40.000 e 50.000.

A GPU foi a **medição mais instável** do trabalho: desvio padrão de 0,13 s sobre 0,71 s no maior volume, contra 0,05 s do numba paralelo.

A margem de 7,5% que separa as duas é menor que a própria variação da GPU entre repetições.

A primeira thread extra paga bem; as seguintes, não

Varrendo o número de threads com 50.000 imagens, o speedup satura em 1,68x e a eficiência paralela cai de 100% para 21%.



THREADS	TEMPO	SPEEDUP	EFICIÊNCIA
1	1,364s	1,00x	100%
2	0,935s	1,46x	73%
4	0,868s	1,57x	39%
8	0,811s	1,68x	21%

Leitura prática: 2 a 4 threads capturam quase todo o ganho disponível. Reservar 8 núcleos para este pipeline desperdiça máquina.

\$ Limite teórico

54% do tempo é serial, e a lei de Amdahl limita o ganho a 1,9x

O speedup de 1,68x com 8 threads não é um resultado ruim de implementação: é o que a fração serial do pipeline permite.

A CONTA

$\text{speedup}(8) = 1,68x$

→ fração serial $\approx 54\%$

→ teto = 1,9x

Com 54% do tempo fora do prange, nenhum número de threads leva o ganho acima de 1,9x.

Três causas se somam: a fração serial do pipeline (leitura e cópia), o trabalho minúsculo por imagem (32×32 pixels não amortizam bem o custo de sincronização) e o fato de as 8 threads serem lógicas, não núcleos físicos.

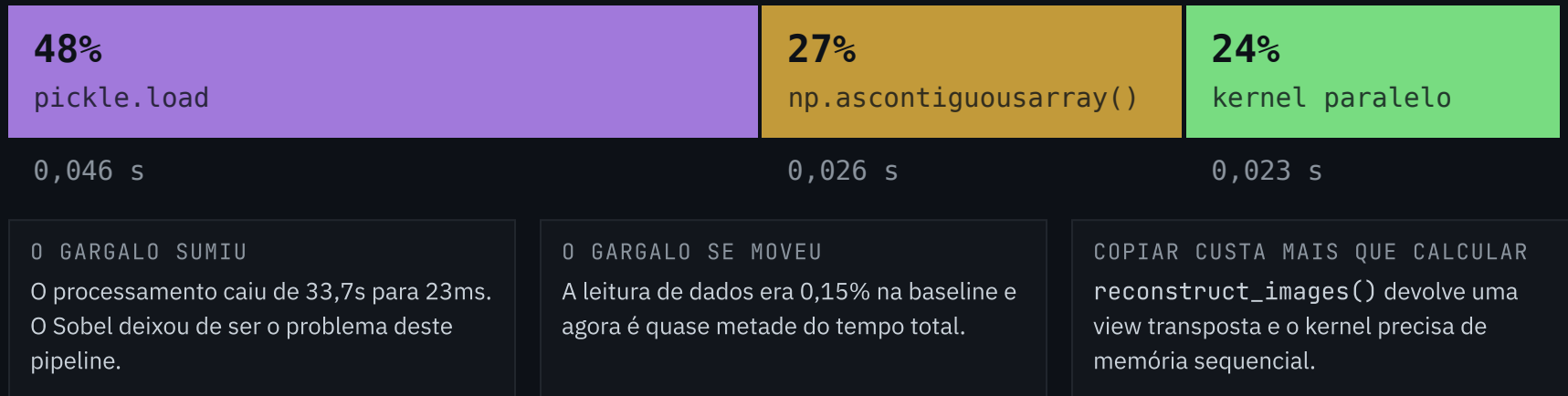
A consequência prática é o slide anterior: o ganho disponível está nas primeiras threads, e a eficiência despenca depois delas.

A consequência de projeto é o slide seguinte: para subir o teto, é preciso encolher a parte serial, não adicionar threads.

\$ Perfil da Otimização Final

Na versão vencedora, o kernel é a menor fatia do tempo

Perfil aquecido de 5.000 imagens: o pipeline inteiro custa 0,095s, e três quartos disso acontecem antes do cálculo.



Pickle e cópia são ambos seriais e rodam fora do prange: somados, são exatamente a fração serial que Amdahl aponta como teto do ganho por threads.

\$ Conclusão

O numba paralelo é a melhor escolha em CPU, e a escolha depende do regime de uso

319x

numba sobre a baseline

1,37x

prange sobre o numba serial

7,5%

o que ainda separa a CPU da GPU

CENÁRIO	ESCOLHA	MOTIVO
script de execução única, volume pequeno, sem cache	vetorizado	não paga compilação por uma execução só
serviço ou lote recorrente, só CPU	numba paralelo, 2 a 4 threads	melhor tempo em CPU, JIT amortizado, código próximo do original
máquina compartilhada, vários lotes simultâneos	numba serial	libera núcleos: a eficiência paralela cai rápido
volumes muito acima de 50.000, com GPU já no fluxo	GPU	a vantagem cresce com o volume, se o hardware já está pago

Lições Aprendidas

01

A melhor otimização é a que ataca o **gargalo apontado pelo profiling**. As técnicas que reduzem o custo por operação (vetorização e compilação) ganharam de longe das que apenas multiplicam unidades de execução.

02

A medição precisa **separar custo de setup de custo de processamento**. Sem essa separação a conclusão se inverte: medir mal é uma forma de errar tão eficiente quanto otimizar a coisa errada.

03

O gargalo se move, e vale perseguir o próximo. Foi ao investigar por que a GPU escalava melhor que surgiu a otimização mais eficiente do projeto e, depois dela, o pipeline passou a gastar quase metade do tempo lendo pickle. A próxima rodada de otimização não seria sobre o Sobel.

\$ Limitações e próximos passos

O que este trabalho não mediu, e o que mediria a seguir

PRÓXIMO PASSO MAIS ÓBVIO

Fazer `reconstruct_images()` devolver um array já contíguo eliminaria a cópia de 26ms e reduziria a fração serial. É hipótese a medir, não resultado.

MARGENS PEQUENAS

As versões medidas nas duas sessões ficaram de 14% a 41% mais lentas na segunda, na mesma máquina. Margens como os 7,5% entre CPU e GPU pedem mais repetições antes de serem tratadas como definitivas.

CARGA DE DADOS

Com o pickle ocupando 48% do tempo, o alvo seguinte é o formato: `.npy`, `memmap` ou leitura paralela dos cinco batches.

ESCOPO DO EXPERIMENTO

Uma máquina e um par CPU/GPU específico: o ponto de cruzamento não é uma constante. Também não foi testada a mesma comparação com imagens maiores, onde a GPU deve dominar com clareza.

\$ obrigada

Obrigada

Perguntas?

github.com/MaluSevilha/projeto-final-compdes

Código, CSVs de benchmark e arquivos de profiling estão no repositório do projeto.

Giulia Roggero

Maria Luiza Sevilha Seraphico

Thais Sabaini